What is NewSQL?

NewSQL is a class of relational database management systems (RDBMS) designed to provide the scalability of NoSQL systems for online transaction processing (OLTP) read-write workloads while maintaining the ACID guarantees (Atomicity, Consistency, Isolation, Durability) of traditional database systems. It delivers a modern solution where businesses can handle large volumes of data in real-time, without having to sacrifice consistency or reliability.

History

The term NewSQL was coined by 451 Research analyst Matt Aslett in 2011, and has been adopted by a number of vendors who do not fit into the traditional RDBMS mold, yet also diverge from the NoSQL movement. NewSQL systems are designed to operate in distributed clusters like NoSQL databases, but they present a relational model and support SQL query semantics.

Functionality and Features

NewSQL databases offer a range of features and functions aimed at preserving SQL's robustness while enhancing scalability and performance. These include:

Full support for SQL: The ability to process complex SQL queries, a feature that's not typically available in NoSQL databases.

Scalability: The ability to scale horizontally across multiple nodes for high-volume traffic while maintaining high transaction rates.

Fault Tolerance: The inclusion of mechanisms to prevent data loss in case of system failure.

Distributed Transactions: Support for ACID transactions across distributed databases.

Built-In Machine Learning: Some NewSQL databases incorporate machine learning functionality directly into the database.

Architecture

The architecture of NewSQL databases is typically distributed and scales horizontally. The system architecture is composed of multiple layers including a SQL layer and a distributed transactional storage layer. The SQL layer processes queries and transactions, while the storage layer manages distributed data access and controls concurrency and recovery.

Benefits and Use Cases

NewSQL fills the gap between traditional RDBMS and NoSQL databases, making it suitable for a variety of use cases. It is particularly well-suited for applications that require high transactional throughput along with strict consistency, such as financial systems and online retail applications. It also benefits big data analytics by providing a scalable database layer that supports complex SQL queries.

Challenges and Limitations

While NewSQL databases offer many advantages, they are not without limitations. For instance, NewSQL services can be complex to administer due to their distributed nature. Also, as a newer technology, they may not yet offer the complete suite of tools and functionalities provided by mature RDBMS platforms.

Integration with Data Lakehouse

NewSQL can be an instrumental component in a data lakehouse setup, as it enables transactionally consistent operations on large volumes of data. This supports the lakehouse

model of providing a unified platform for various workloads such as data warehousing and machine learning.

Security Aspects

In terms of security, NewSQL systems generally incorporate traditional database security measures, including user authentication and authorization, data encryption, and activity logging. However, the specific security features may vary based on the individual product.

Performance

NewSQL database systems aim to provide the high performance of NoSQL systems while maintaining the ACID properties of traditional SQL. This efficiency can be particularly beneficial in applications requiring high-volume, real-time data processing.

What is the main advantage of NewSQL over traditional SQL?

The key advantage is scalability. NewSQL databases are designed to scale horizontally across multiple servers while maintaining high transaction processing speeds and full ACID compliance.

How is NewSQL different from NoSQL?

Unlike NoSQL, NewSQL databases support relational data models and SQL query semantics. They still offer the scalability and performance of NoSQL systems while maintaining a higher degree of consistency.

Can NewSQL databases replace traditional RDBMS?

While NewSQL databases can provide many of the same guarantees as traditional RDBMS, they are not necessarily a replacement but rather a modern supplement to traditional solutions. The choice between the two depends greatly on the specific use case, workload requirements, and complexity of queries.

Glossary

ACID: A set of properties that guarantee database transactions are processed reliably.

SQL: Structured Query Language, a standard language for managing and manipulating databases.

NoSQL: Non-relational databases that scale horizontally and are designed to store, distribute, and access large amounts of data across many servers.

Database Scalability: The ability of a database system to handle an increasing amount of work by adding resources.

Data Lakehouse: An architecture that combines qualities of data lakes and data warehouses in a single platform.

Difference between NoSQL and NewSQL

1. NoSQL:

The term NoSQL is categorizing databases as descriptive as " No-SQL ". NoSQL is a comprehensive category of databases that are developed to overcome the problems generated by SQL databases. They are referred to as schema less documents which store the data in documents, graph, key-value, and non-ordered fashion.

Advantages of NoSQL:

They scale better than traditional systems when there is a need for dynamic behavior.

These systems are better optimized for non-relational data.

Allows performing schema-on-write operations.

Disadvantages of NoSQL:

The system built with NoSQL is fundamentally non-transactional.

The volume of data created is huge and does not offer any traditional database capability.

It does not follow consistency when multiple transactions are performed simultaneously.

2. NewSQL:

The term NewSQL categorizes databases that are the combination of relational model with the advancement in scalability, flexibility with types of data. These databases focus on the features which are not present in NoSQL, which offers a strong consistency guarantee. This covers two layers of data one relational one and a key-value store.

Advantages of NewSQL:

It introduces new implementation to traditional relational databases.

It brings together the advantages of SQL and NoSQL.

It is easy to migrate between the type and needs of the user.

Disadvantages of NewSQL:

They offer partial access to rich traditional systems.

It may cause a problem in-memory architecture for exceeding volumes of data.

The core foundation of such databases is relational systems which make it tricky to understand.

Difference between NoSQL and NewSQL :

S.No	NoSQL	NewSQL
1.	NoSQL is a schema-free database.	NewSQL is schema-fixed as well as a schema-free database.
2.	It is horizontally scalable.	It is horizontally scalable.
3.	It possesses automatically high-availability.	It possesses built-in high availability.
4.	It supports cloud, on-disk, and cache storage.	It fully supports cloud, on-disk, and cache storage.
5.	It promotes CAP properties.	It promotes ACID properties.
6.	Online Transactional Processing is not supported.	Online Transactional Processing is fully supported.
7.	There are low-security concerns.	There are moderate security concerns.
8.	Use Cases: Big Data, Social Network Applications, and IOT.	Use Cases: E-Commerce, Telecom industry, and Gaming.
9.	Examples : DynamoDB, MongoDB, RaveenDB etc.	Examples : VoltDB, CockroachDB, NuoDB etc.

Functions	RDBMS	NoSQL	NewSQL
SQL	Supported	Not supported	Supported
Machine dependency	Singe machine	Multi- machine/Distributed	Multi- machine/Distributed
DBMS type	Relational	Non- relational	Relational
Schema	Table	Key-value, column store, document store	Both
Storage	On disk + cache	On disk + cache	On disk + cache
Properties support	ACID	CAP through BASE	ACID
Horizontal scalability	Not supported	Supported	Supported
Query Complexity	Low	High	Very High
Security concern	Very high	Low	Low
Big volume	Less performance	Fully supported	Fully supported
OLTP	Not fully supported	Supported	Fully supported
Cloud support	Not fully supported	Supported	Fully supported

CockroachDB Architecture Overview

CockroachDB was designed to create the source-available database we would want to use: one that is both scalable and consistent. Developers often have questions about how we've achieved this, and this guide sets out to detail the inner workings of the cockroach process as a means of explanation.

However, you definitely do not need to understand the underlying architecture to use CockroachDB. These pages give serious users and database enthusiasts a high-level framework to explain what's happening under the hood.

Goals of CockroachDB

CockroachDB was designed to meet the following goals:

- Make life easier for humans. This means being low-touch and highly automated for operators and simple to reason about for developers.
- Offer industry-leading consistency, even on massively scaled deployments. This means enabling distributed transactions, as well as removing the pain of eventual consistency issues and stale reads.
- Create an always-on database that accepts reads and writes on all nodes without generating conflicts.
- Allow flexible deployment in any environment, without tying you to any platform or vendor.
- Support familiar tools for working with relational data (i.e., SQL).

With the confluence of these features, we hope that CockroachDB helps you build global, scalable, resilient deployments and applications.

Overview

CockroachDB starts running on machines with two commands:

- cockroach start with a --join flag for all of the initial nodes in the cluster, so the process knows all of the other machines it can communicate with.
- cockroach init to perform a one-time initialization of the cluster.

Once the CockroachDB cluster is initialized, developers interact with CockroachDB through a PostgreSQL-compatible SQL API. Thanks to the symmetrical behavior of all nodes in a cluster, you can send SQL requests to any node; this makes CockroachDB easy to integrate with load balancers.

After receiving SQL remote procedure calls (RPCs), nodes convert them into key-value (KV) operations that work with our distributed, transactional key-value store.

As these RPCs start filling your cluster with data, CockroachDB starts algorithmically distributing your data among the nodes of the cluster, breaking the data up into chunks that we call ranges. Each range is replicated to at least 3 nodes by default to ensure survivability. This ensures that if any nodes go down, you still have copies of the data which can be used for:

- Continuing to serve reads and writes.
- Consistently replicating the data to other nodes.

If a node receives a read or write request it cannot directly serve, it finds the node that can handle the request, and communicates with that node. This means you do not need to know where in the cluster a specific portion of your data is stored; CockroachDB tracks it for you, and enables symmetric read/write behavior from each node.

Any changes made to the data in a range rely on a consensus algorithm to ensure that the majority of the range's replicas agree to commit the change. This is how CockroachDB achieves the industry-leading isolation guarantees that allow it to provide your application with consistent reads and writes, regardless of which node you communicate with.

Ultimately, data is written to and read from disk using an efficient storage engine, which is able to keep track of the data's timestamp. This has the benefit of letting us support the SQL standard AS OF SYSTEM TIME clause, letting you find historical data for a period of time.

While the high-level overview above gives you a notion of what CockroachDB does, looking at how CockroachDB operates at each of these layers will give you much greater understanding of our architecture.

Layers

At the highest level, CockroachDB converts clients' SQL statements into key-value (KV) data, which is distributed among nodes and written to disk. CockroachDB's architecture is manifested as a number of layers, each of which interacts with the layers directly above and below it as relatively opaque services.

The following pages describe the function each layer performs, while mostly ignoring the details of other layers. This description is true to the experience of the layers themselves, which generally treat the other layers as black-box APIs. There are some interactions that occur between layers that require an understanding of each layer's function to understand the entire process.

Layer	Order	Purpose
SQL	1	Translate client SQL queries to KV operations.
Transactional	2	Allow atomic changes to multiple KV entries.
Distribution	3	Present replicated KV ranges as a single entity.
Replication	4	Consistently and synchronously replicate KV ranges across many nodes. This layer also enables consistent reads using a consensus algorithm.
Storage	5	Read and write KV data on disk.

Consistency

The requirement that a transaction must change affected data only in allowed ways. CockroachDB uses "consistency" in both the sense of ACID semantics and the CAP theorem, albeit less formally than either definition.

Isolation

The degree to which a transaction may be affected by other transactions running at the same time. CockroachDB provides the SERIALIZABLE and READ COMMITTED isolation levels. For more information, see Isolation levels.

Consensus

The process of reaching agreement on whether a transaction is committed or aborted. CockroachDB uses the Raft consensus protocol. In CockroachDB, when a range receives a write, a quorum of nodes containing replicas of the range acknowledge the write. This means your data is safely stored and a majority of nodes agree on the database's current state, even if some of the nodes are offline.

When a write does not achieve consensus, forward progress halts to maintain consistency within the cluster.

Replication

The process of creating and distributing copies of data, as well as ensuring that those copies remain consistent. CockroachDB requires all writes to propagate to a quorum of copies of the data before being considered committed. This ensures the consistency of your data.

Transaction

A set of operations performed on a database that satisfy the requirements of ACID semantics. This is a crucial feature for a consistent system to ensure developers can trust the data in their database. For more information about how transactions work in CockroachDB, see Transaction Layer.

Transaction contention

A state of conflict that occurs when:

- A transaction is unable to complete due to another concurrent or recent transaction attempting to write to the same data. This is also called *lock contention*.
- A transaction is automatically retried because it could not be placed into a serializable ordering among all of the currently executing transactions. This is also called

a *serialization conflict*. If the automatic retry is not possible or fails, a *transaction retry error* is emitted to the client, requiring a client application running under SERIALIZABLE isolation to retry the transaction.

Steps should be taken to reduce transaction contention in the first place.

Multi-active availability

A consensus-based notion of high availability that lets each node in the cluster handle reads and writes for a subset of the stored data (on a per-range basis). This is in contrast to *active-passive replication*, in which the active node receives 100% of request traffic, and *active-active* replication, in which all nodes accept requests but typically cannot guarantee that reads are both up-to-date and fast.

User

A SQL user is an identity capable of executing SQL statements and performing other cluster actions against CockroachDB clusters. SQL users must authenticate with an option permitted on the cluster (username/password, single sign-on (SSO), or certificate). Note that a SQL/cluster user is distinct from a CockroachDB Cloud organization user.

CockroachDB architecture terms

Cluster

A group of interconnected CockroachDB nodes that function as a single distributed SQL database server. Nodes collaboratively organize transactions, and rebalance workload and data storage to optimize performance and fault-tolerance.

Each cluster has its own authorization hierarchy, meaning that users and roles must be defined on that specific cluster.

A CockroachDB cluster can be run in CockroachDB Cloud, within a customer Organization, or can be self-hosted.

Node

An individual instance of CockroachDB. One or more nodes form a cluster.

Range

CockroachDB stores all user data (tables, indexes, etc.) and almost all system data in a sorted map of key-value pairs. This keyspace is divided into contiguous chunks called *ranges*, such that every key is found in one range.

From a SQL perspective, a table and its secondary indexes initially map to a single range, where each key-value pair in the range represents a single row in the table (also called the *primary index* because the table is sorted by the primary key) or a single row in a secondary index. As soon as the size of a range reaches the default range size, it is split into two ranges. This process continues for these new ranges as the table and its indexes continue growing.

Replica

A copy of a range stored on a node. By default, there are three replicas of each range on different nodes.

Leaseholder

The replica that holds the "range lease." This replica receives and coordinates all read and write requests for the range.

For most types of tables and queries, the leaseholder is the only replica that can serve consistent reads (reads that return "the latest" data).

The leaseholder is always the same replica as the Raft leader, except briefly during lease transfers. For more information, refer to Leader leases.

Raft protocol

The consensus protocol employed in CockroachDB that ensures that your data is safely stored on multiple nodes and that those nodes agree on the current state even if some of them are temporarily disconnected.

Raft leader

For each range, the replica that is the "leader" for write requests. The leader uses the Raft protocol to ensure that a majority of replicas (the leader and enough followers) agree, based on their Raft logs, before committing the write. The Raft leader is always the same replica as the leaseholder. For more information, refer to Leader leases.

Raft log

A time-ordered log of writes to a range that its replicas have agreed on. This log exists on-disk with each replica and is the range's source of truth for consistent replication.

Fault Tolerance Mechanisms in NewSQL:-

NewSQL databases ensure high fault tolerance by combining ACID-compliant, traditional SQL relational models with distributed, horizontal scaling. They achieve resilience through automatic data replication, node sharding, and consensus protocols (like Paxos or Raft), allowing systems to survive node, data center, or region failures without losing data consistency.

Key Fault Tolerance Mechanisms in NewSQL

- **Distributed Consensus Protocols:** Systems like CockroachDB and Google Spanner utilize protocols such as Raft or Paxos to manage replication, ensuring that even if a minority of nodes fail, the system continues to function with a consistent state.
- **Automatic Replication & Sharding:** Data is automatically divided (sharded) and replicated across multiple nodes, racks, or data centers, providing automatic failover if a node goes down.
- **ACID Compliance in Distributed Systems:** NewSQL ensures strong consistency, meaning transactions are guaranteed, preventing data corruption during network partitions or node failures.
- **Geographical Resilience:** Advanced NewSQL solutions, such as Google Spanner, offer geographical fault tolerance, maintaining availability and consistency even during an entire data center outage.
- **Self-Healing:** Distributed architectures detect failures and automatically re-replicate data from surviving nodes to restore high availability.

Examples of Fault-Tolerant NewSQL Systems

- **Google Spanner:** Uses TrueTime for global consistency and high fault tolerance.
- **CockroachDB:** Built specifically for horizontal scalability and surviving node or data center failures.

- **YugabyteDB:** Focuses on distributed SQL with strong consistency across geographically distributed nodes.
- NewSQL bridges the gap between traditional RDBMS reliability and NoSQL scalability, making it suitable for mission-critical applications where data loss is not an option.

NewSQL systems provide the high-performance ACID transactions of traditional relational databases (RDBMS) combined with the horizontal scalability of NoSQL, typically achieved through distributed, shared-nothing architectures and automated partitioning. They allow systems to handle massive OLTP workloads and data volumes across multiple nodes, offering linear scaling without sacrificing data consistency.

Performance and Scalability Highlights

- **Horizontal Scaling:** Unlike RDBMS, which scales vertically (bigger servers), NewSQL scales horizontally by adding more nodes to a cluster, allowing for linear performance growth.
- **ACID Compliance:** NewSQL maintains strong consistency (ACID) across distributed systems, which is vital for financial and transactional applications, often lacking in traditional NoSQL.
- **Distributed Architecture:** Data is automatically partitioned (sharded) and distributed across nodes, preventing bottlenecks and handling high-velocity data ingestion.
- **High Performance & Availability:** Built for high-traffic, real-time analytics and fast transactional processing, NewSQL systems like Google Spanner and CockroachDB offer high fault tolerance.

Usage Examples

- **Financial & Payment Systems:** CockroachDB and similar systems are used for global, transactional data requiring strong consistency.
- **E-commerce & Retail:** VoltDB is used for real-time inventory management and personalized offers in high-volume, low-latency environments.
- **Global Applications:** Google Spanner handles global consistency for applications demanding high availability across regions.
- **IoT and Real-Time Data:** Distributed SQL databases, such as TiDB, manage and analyze growing IoT data streams.

Synonyms/Key Characteristics

- **Distributed SQL:** Often used interchangeably, emphasizing SQL functionality on distributed systems.
- **Elastic Scalability:** Ability to seamlessly scale in/out by adding nodes.
- **HTAP (Hybrid Transactional/Analytical Processing):** Some NewSQL systems support both fast transactions and complex analytical queries.

Q. **Describe the future trends in NoSQL/NewSQL and how they bridge**

the RDBMS gap.

As of 2026, the database landscape has moved beyond the simple "SQL vs. NoSQL" debate, entering an era of intelligent convergence where NoSQL adopts stronger consistency, NewSQL provides horizontal scale for relational data, and both embrace AI-native capabilities.

The core future trend is polygamous persistence, where specialized databases (graph, document, time-series) coexist within a unified, cloud-native ecosystem.

Future Trends in NoSQL/NewSQL (2026-2030)

- **AI-Native and Vector-First Architectures:** NoSQL databases (e.g., MongoDB, Cassandra, Redis) are increasingly embedding vector search directly into their engines to support AI retrieval-augmented generation (RAG) pipelines. This allows databases to store high-dimensional embeddings alongside operational data, facilitating real-time semantic search.
- **Rise of Distributed SQL (NewSQL):**

NewSQL solutions like CockroachDB, TiDB, and YugabyteDB are becoming the standard for mission-critical applications that require massive horizontal scale *and* ACID compliance. These databases use distributed consensus algorithms (Raft/Paxos) and hybrid logical clocks to provide global consistency.

- **Edge Computing and Geo-Distribution:**

Databases are moving closer to the user to reduce latency. Technologies like Turso and CockroachDB enable replicating data to edge nodes globally, ensuring sub-millisecond query response times.

- **Serverless and Consumption Pricing:** The shift toward serverless databases (e.g., MongoDB Atlas, DynamoDB, PlanetScale) continues, offering automatic scaling and pay-per-use billing, reducing operational overhead.
- **Multi-Model Convergence:** Rather than managing multiple databases, organizations are adopting multi-model platforms that support document, key-value, graph, and relational models within a single engine.
- **Autonomous Operations:** Databases are increasingly incorporating AI for self-healing, auto-tuning, and intelligent indexing to optimize performance without human intervention.

How NewSQL Bridges the RDBMS Gap

NewSQL serves as a direct, modern evolution of RDBMS designed to tackle the scaling limitations of traditional systems like MySQL or PostgreSQL, while preserving their core strengths.

RDBMS Limitation	NoSQL Solution	NewSQL Bridge (How they bridge the gap)
Vertical Scaling Only	Horizontal Scaling	**Distributed Architecture:** Shared-nothing architecture allows nodes to be added linearly to scale compute/storage.
ACID Compliance Constraints	Eventual Consistency	**Distributed ACID Transactions:** Uses consensus protocols (Raft/Paxos) to ensure strict consistency across distributed nodes.
Complex Queries Lack Support	Limited Joins	**Full SQL Compatibility:** Supports complex JOINs, aggregations, and standard SQL syntax, often via MySQL/PostgreSQL compatibility.
Schema Rigidity	Flexible Schema	**Hybrid Models:** Many NewSQL systems now support JSONB or schemaless document structures alongside relational tables.

Summary of Key Technologies 2026

- **Distributed SQL:** CockroachDB , TiDB, YugabyteDB, Google AlloyDB.
- **NoSQL Leaders:** MongoDB (Document), Redis (In-memory/Vector), Cassandra (Wide-column), Neo4j (Graph).
- **Edge/Serverless SQL:** Turso, PlanetScale.

The future of data management is not about one type winning, but rather about using NewSQL for strict transactional consistency and NoSQL for extreme flexibility and speed, both managed within unified, serverless cloud environments.